CS 541: APPLIED MACHINE LEARNING
CHALLENGE REPORT

Name: Chris Mary Benson
ID: U56085268

CONTENT

1. Introduction 3

2. Models 4
   2.1 Part 1: Training a CNN model on CIFAR 100 from scratch 4
   2.2 Part 2: Finetuning a Pre-Trained CNN on CIFAR 100 7
   2.3 Part 3: Finetuning a Pre-Trained Transformer on CIFAR 100 9

3. Conclusion 14
4. References 15

1. INTRODUCTION

Disclaimer: This is in regards to the coding involved in all three models, I have used AI assistance (Claude + ChatGPT), to find more ways for strategic positioning of certain blocks, for help in finding the most optimal values for the hyper parameters and seeing if there are more efficient ways to structure my configurations.

This is how I structured my prompts to AI:

- Put my code, asked if it was the correct syntax, and asked if there were any better optimizations.
- Asked for optimal values for hyperparameters
- Better regularization techniques
- Ways to finetune pre-trained models

The code for which I used AI assistance has been labelled as such (through comments) in the Colab notebooks. These notebooks also contain further detailed comments of the models I have tested with.

2. MODELS

2.1 Part 1: Training a CNN model on CIFAR 100 from scratch

I explored several iterations of the CNN architecture, working with different configurations to improve the robustness of the model to the hidden test set which contained distortions.

Trial 1:

My first modification introduced residual blocks, cosine annealing learning rate scheduling and global average pooling before the classifier, and an initial learning rate of 3e-3.

This model reached approximately 90% training accuracy within 30 epochs. However, validation accuracy improved slowly and plateaued near 60%, which suggested that the model was overfitting despite fast convergence.

On the hidden test, the model achieved a hidden test accuracy of 38%, indicating that the model was not robust to varying distorted images yet.

Trial 2:

To improve the robustness of the model, I added more augmentations to the data; this included colour jitter and gaussian blur alongside the random crop and horizontal flip.

Alongside the data augmentations, I also added dropout regularization of 0.3 and switched to a SGD optimizer instead of AdamW.

Although training accuracy decreased slightly to 88% after 100 epochs, validation accuracy improved to 66%. This version achieved 71% test accuracy, showing that sacrificing some training performance helped improve generalization.

On the hidden test set, this achieved an accuracy of 43.8%, suggesting that the model was improving in robustness.

Trial 3:

Next, I introduced RandAugment in the data augmentations, and increased both the learning rate and weight decay.

Training accuracy reached around 87%, while validation accuracy increased to 68%. Clean test accuracy improved further to 75%, indicating that stronger augmentation and regularization were beneficial.

Trial 4:

I then repositioned the residual blocks earlier in the CNN model (before the pooling layers), added label smoothing

I also extended the training to 120 epochs, and increased the dropout to 0.3. This version achieved approximately 91% training accuracy and 69% validation accuracy. Clean test accuracy reached 76.8%, while hidden test accuracy increased significantly (+13% increase from the previous model) to 55.6%, suggesting improved robustness on the distorted data.

Final Model:

In the final version, I redesigned the residual block configuration by adding more residual blocks to emulate a Resnet style model and modified the learning rate scheduler to include a warmup phase before decay, to see the effects of no weight decay in the parameters of the batch normalization layers [1].

I also trained the model for 150 epochs. Although training accuracy was slightly lower at 89%, validation accuracy improved to 71%. Clean test accuracy remained at 76.8%.

This model outperformed the previous version with a 10% increase in the hidden test accuracy (**64.9%**), which was the best overall result I achieved for this CNN model on distorted data.

2.1.2 Findings

From these multiple trial and error experiments that I ran, I found that the most effective improvements came from:

- Increasing the model depth with additional residual blocks
- Applying stronger regularization through dropout, weight decay, and label smoothing
- Using aggressive data augmentation such as AugMix and RandAugment
- Improving optimization with learning rate, warmup (combining two schedulers)

Most importantly, the hidden test accuracy improved from 38% to 64.8%, showing that later experiments were much more successful at improving generalization to distorted images rather than focusing on just the training and validation accuracy.

Additions to the CNN Model	***Best Training Accuracy***	***Best Validation Accuracy***	***Test Accuracy***	***Hidden Test Accuracy***
Residual block, more conv layers, 30 epochs	90%	60%	-	38%
More data augmentations + SGD optimizer + dropout, 100 epochs	88%	66%	71%	43.8%
Minor changes: Added RandAugment to the data augmentations	87%	68%	75%	
Repositioning residual blocks within the CNN model + label smoothing + increased dropout, 120 epochs	91%	69%	76.8%	55.6%
Final Model: Redesigning the residual block to include drop paths + redesigning the CNN model to simulate Resnet architecture + removed weight decay for batch norm layers, 150 epochs	**89%**	**71%**	**76.8%**	**64.9%**

Table 1: Comparing Different Experiments for CNN

***Note** - For each version in the above table, I have done trial and error with different learning rates and weight decay values. Each version is building on the previous one, unless mentioned otherwise.

2.2 Part 2: Finetuning a Pre-Trained CNN on CIFAR 100

For the next part, I did some research on what pre-trained CNN models I could experiment with. I focused mainly on finding models that are proven to be robust. I experimented with the following ones:

- Resnet50
- ConvNext

I will go in detail about Resnet50 below.

Resnet50

I chose ResNet50 because of its structure containing residual connections which allow for deep feature learning while avoiding vanishing gradients, and its ImageNet pretraining that transfers well to CIFAR-100.

To adapt ResNet50 for CIFAR-100's 32x32 images, I replaced the first convolutional layer with a 3x3 kernel instead of the original 7x7, and removed the max pooling layer. This prevents too much downsampling on small images which would destroy spatial information early in the network. I also replaced the final classification head with a dropout classifier (0.5 and 0.3 dropout rates) with a 512-dimensional hidden layer to better regularize for 100 classes.

For robustness to OOD corruptions, I used AugMix as my primary augmentation strategy since it is designed to improve model robustness to unseen corruptions by mixing augmented views of images.

I combined this with other augmentations (used in part 1) including random horizontal flips, random crops, color jitter, random grayscale, and Gaussian blur, followed by random erasing after normalization.

I trained with SGD, a weight decay of 5e-4 applied only to non-BatchNorm parameters, and label smoothing of 0.2 (similar to my previous from scratch model) to prevent overconfidence on 100 classes. I used a warmup schedule for the first 5 epochs followed by cosine annealing decay to ensure stable convergence of the pretrained weights.

With 100 epochs, **Hidden Test Accuracy: 75%**, Train Accuracy: 80%, Validation Accuracy: 69%, Clean Test Accuracy: 83.3% (Refer Figure 1.)

I also tried other experiments:- mainly adjusting the learning rate and weight decay, as well as playing around with the augmentations; but the hidden test accuracy did not improve, it stayed in the range of 69 - 71%.

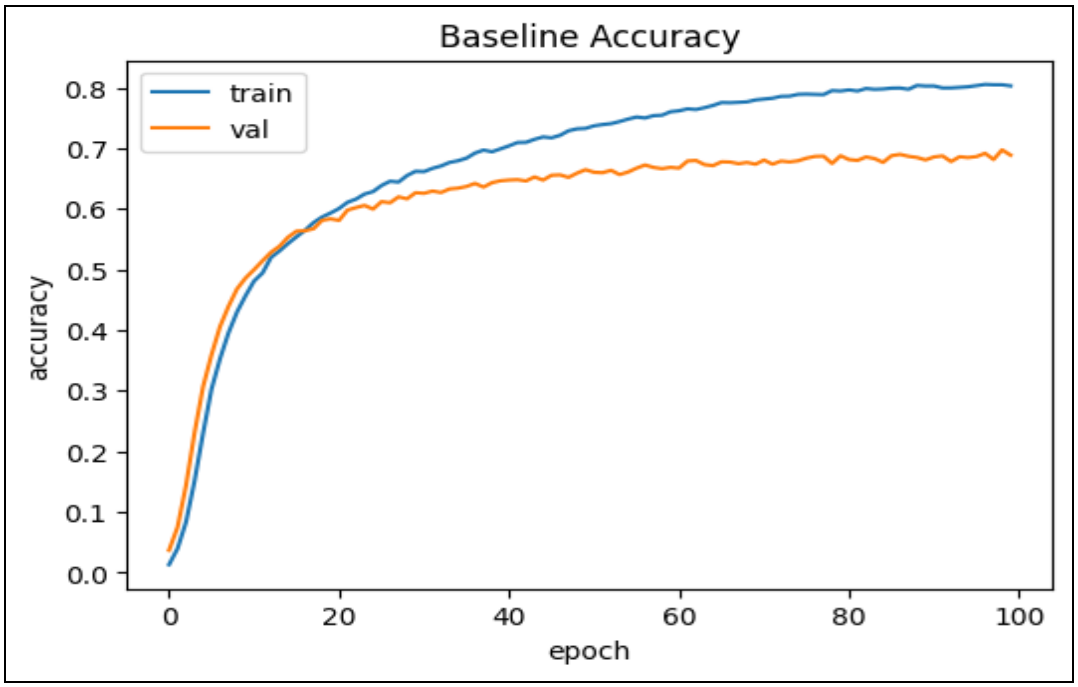


Figure 1: Train and Validation Accuracy for Resnet50

2.2.2 Findings

The fine-tuned ResNet50 achieved strong performance on clean data (83.3% test accuracy), showing effective transfer learning on CIFAR-100. However, the gap between training (80%) and validation accuracy (69%) indicates some overfitting.

The lower hidden test accuracy (75%) compared to clean test accuracy suggests limited robustness to distribution shifts, even with AugMix. Overall, while ResNet50 performs well as a baseline, its robustness to unseen corruptions remains a key limitation.

2.3 Part 3: Finetuning a Pre-Trained Transformer on CIFAR 100

For Part 3, I experimented with three different pre-trained transformer-based models: DeiT, Swin Transformer, and DINOv2. I used the small variants of each model because the base versions required significantly more memory and computational resources, which often led to session crashes during training.

(i) Deit

DeiT is a Vision Transformer-based architecture, and I initially chose this because it is designed to perform well even on relatively smaller datasets by learning global relationships between image patches using self-attention, rather than relying on local convolutional operations like CNNs.

I resized all input images to 224 × 224 to match the expected input size of the DeiT model. The images were normalized using the given dataset mean and standard deviation values during both training and evaluation to ensure stable convergence.

For training, I used a batch size of 64 and trained the model for 50 epochs. I selected a learning rate of 3e-4, which is commonly used for transformer-based architectures, and applied a weight decay of 0.05 to improve generalization and reduce overfitting.

I used SoftTargetCrossEntropy as the loss function since I applied mixup augmentation during training.

In addition to the data augmentation used in my earlier CNN experiments, I applied a stronger and more diverse augmentation pipeline for training the transformer-based model. I specifically added random rotation and random affine transformations in this stage, since transformer-based models like DeiT can benefit from stronger geometric variations and learn more robust global representations.

I achieved a training accuracy of 65%, a validation accuracy of 87% and a hidden test accuracy of 75%, which suggests that model was able to generalize well to the dataset.

I also trained a second time, for 100 epochs, but the model did not benefit from this, the training and validation accuracies plateaued, giving the same accuracies for all three (training, validation and hidden).

(ii) Swin

I tried to explore another transformer that is already known for strong performance and robustness. Swin Transformer introduces a shifted window-based self-attention mechanism, which makes it more effective at capturing both local and global features compared to standard Vision Transformers.

However, in my experiments, Swin Transformer did not outperform my DeiT-based model. While Swin showed stable learning behavior and good generalization, it consistently achieved lower validation and test accuracy compared to DeiT under the same training setup.

But another important note is that I made the data augmentation setup more conservative, reducing mixup configuration values as well as the normal data augmentation ones. I did this because I wanted to observe if the augmentations were being too aggressive, making the model underperform.

My theory turned out to be wrong, since this model trained for 50 epochs did not give a higher accuracy, the hidden test accuracy was 65.8%, a 10% decrease than Deit's model.

(iiI) Dinov2

For the final trial, I used DINOv2, a self-supervised vision transformer pretrained on large-scale image data, and fine-tuned it for 50 epochs on CIFAR-100.

I applied a similar training pipeline as in previous experiments (similar to Deit), with data augmentations and standard optimization settings.

The model achieved strong convergence during training and validation, demonstrating stable learning and good generalization on the CIFAR-100 dataset.

I trained the model using a two-stage fine-tuning strategy to improve the final performance. In the first stage, I froze the backbone of the network and trained only the classification head for 5 epochs. This allowed the model to quickly adapt its final decision layer to the CIFAR-100 dataset without disturbing the pretrained feature representations.

In the second stage, I unfroze the entire model and performed full fine-tuning for 45 epochs. I used different learning rates for different parts of the network, with a higher learning rate for the classification head and lower learning rates for deeper layers such as the transformer blocks and patch embedding layer. This helped preserve useful pretrained features while still allowing task-specific adaptation.

To improve optimization stability, I used a learning rate schedule consisting of a linear warm-up phase followed by cosine annealing decay. The warm-up helped stabilize early training, while cosine annealing allowed for smoother convergence in later epochs. I used AdamW as the optimizer along with a weight decay of 0.05 for regularization.

I achieved a 91% accuracy on the clean test set, and a 79.9% accuracy on the hidden test set, making it the strongest performing model among the three transformer-based architectures I experimented with.

I also tried experimenting with a Test Time Augmentation (TTA) model, which actually changes the way the model is tested at inference time. Basically, instead of giving a single image from the test set to the model, we instead apply more augmentations to the image, get the predictions from the model, and then obtain the average of these predictions. [2] This helps reduce prediction variance and typically improves robustness. In my experiments, TTA provided a slight improvement in accuracy (80.3%), suggesting that aggregating predictions across augmented inputs can enhance the model's generalization.

Overall, DINOv2 provided competitive performance compared to the other transformer-based models, highlighting the effectiveness of self-supervised pretraining for downstream image classification tasks.

Transformer	***Best Training Accuracy***	***Best Validation Accuracy***	***Test Accuracy***	***Hidden Test Accuracy***
Deit small	65%	87%	-	75.5%
Swin small	66%	66%	71%	65.8%
Dinov2 small	**65%**	**87%**	**91%**	**80.3%**

Table 2: Comparing Different Experiments for Part 3

2.2.3 Findings

Class 82 (Sunflower)	**Class 14 (Butterfly)**	**Class 75 (Rabbit)**
		

Table 3: Top 3 Best Performing Classes - Test Set

In table 3, I have recorded the top three performing classes on the clean test set, Sunflower, Butterfly, and Rabbit, share a common trait: they each have highly distinctive visual features that make them easy to differentiate. Sunflower benefits from its unique

bright yellow color, while Butterfly has distinctive features that are rarely shared by other classes. The Rabbit class performs well despite its test image appearing heavily distorted and grainy, suggesting the model has learned robust representations, which is true for DINOv2's due to its self-supervised pretraining and this allows it to recognize the animal's shape.

Class 1 (Goldfish)	Class 29 (Keyboard)	Class 39 (Dinosaur)
		

Table 3: Top 3 Best Performing Classes - Validation Set

In table 3, we now have the top 3 performing classes on the validation set which is the Goldfish, Keyboard, and Dinosaur. These images contain the data augmentations applied to the dataset such as AugMix, Gaussian blur, random rotation, etc. All three images show clear signs of heavy augmentation, yet the model is able to classify them correctly. These heavily augmented versions being the best performing classes suggests that the model has developed robust representations for these categories.

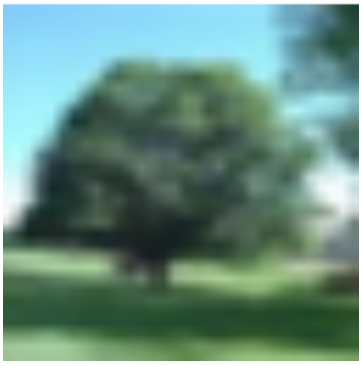
Class 47 (Tree)	Class 98 (Girl)	Class 10 (Candle)
		

Table 5: Top 3 Worst Performing Classes - Test Set

In table 5, we have the worst performing classes on the test set, i.e the Tree, Girl, and Candle. A reason for the tree class could be that it is an ambiguous feature that could be

confused for other types of greenery classes if it is present in CIFAR 100. The Girl class is difficult because CIFAR-100 also contains multiple other classes that are human related (boy, girl, man, woman), hence the model can get confused due to visual similarities. From this, we can conclude that the model would need a better fine tuning strategy where it is easily able to differentiate between classes that have similar features.

Class 11 (Boy)	Class 96 (Plant)	Class 47 (Tree)
		

Table 6: Top 3 Worst Performing Classes - Validation Set

The worst performing classes on the validation set as seen in Table 6 are the Boy, Plant and Tree classes. We can see that the Random Erasing technique has erased some of the features from the boy in the picture and the model failed to recognize that. The Plant and Tre class suffer from a different problem: both are similar classes with high visual similarity to each other (as seen in the test set as well) and the heavy augmentations (color jitter, AugMix) have further distorted the features like leaf shape and trunk structure.

It is observed that the Tree class appears in both the worst performing test and validation classes, confirming it is a difficult class to classify.

3. CONCLUSION

To summarize the results for all the three parts, we can see that the transformer model (with self-supervised) learning performs the best out of all the models experimented with. The ResNet50 model comes second, lagging behind in the accuracy by 9%. The model that performed the least was the CNN model from scratch which is understandable, we would need a very strong CNN configuration, with optimal parameters, and trained for 150+ epochs for the model to be able to generalize well to distortions. It achieved a hidden test accuracy of 65%, which I believe indicates moderate performance on the distorted data.

This trend highlights the advantage of pretraining and transformer architectures in learning more robust and transferable representations, especially on CIFAR-100.

4. REFERENCES

[1]https://discuss.pytorch.org/t/weight-decay-in-the-optimizers-is-a-bad-idea-especially-with-batchnorm/16994/7

[2]https://medium.com/data-science/test-time-augmentation-tta-and-how-to-perform-it-with-keras-4ac19b67fb4d